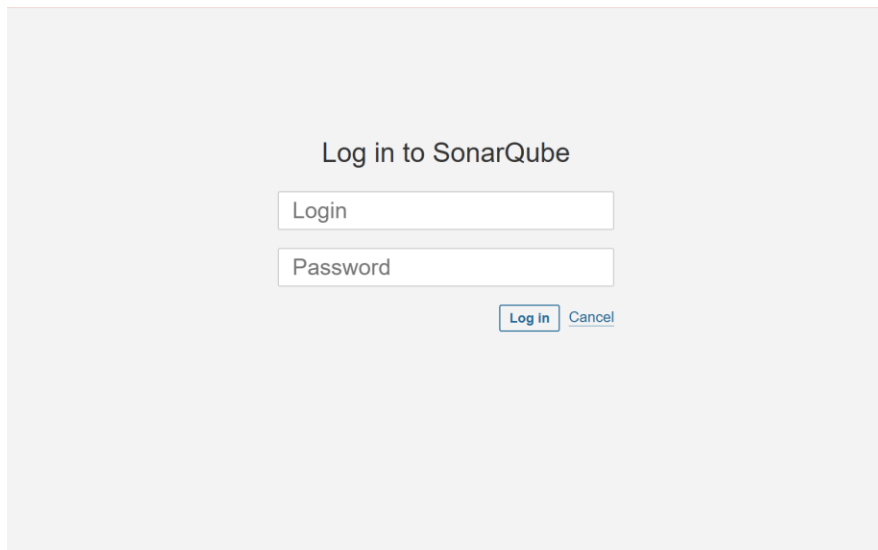
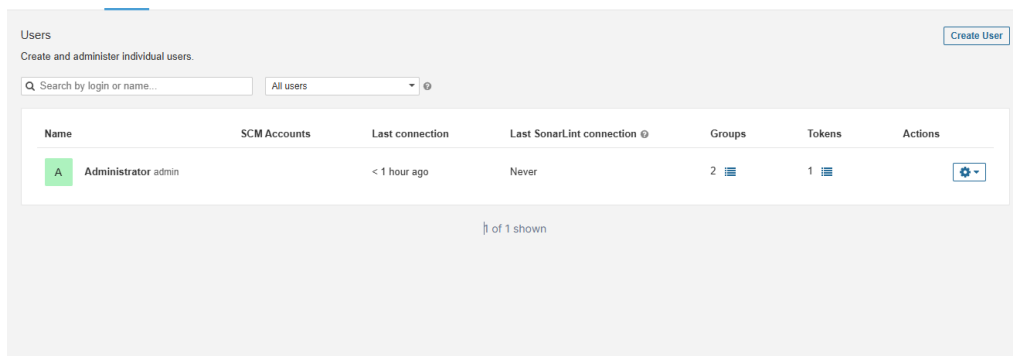


Task 7: Pull and Run SonarQube Docker Image on the EC2 instance



The image shows the SonarQube login page. It has a light gray background with the title "Log in to SonarQube" centered. Below the title are two input fields: "Login" and "Password". At the bottom right of the form are two buttons: "Log in" (in blue) and "Cancel" (in gray).

Task 8: Login to SonarQube using (admin, admin) and change password.



The image shows the "Users" management page in SonarQube. At the top, there's a "Create User" button. Below it, a search bar and a dropdown menu set to "All users". A table lists the users:

Name	SCM Accounts	Last connection	Last SonarLint connection ⓘ	Groups	Tokens	Actions
 Administrator admin		< 1 hour ago	Never	2 	1 	

Below the table, it says "1 of 1 shown".

Task 9: Create SonarQube Webhook to Communicate with Jenkins from SonarQube.

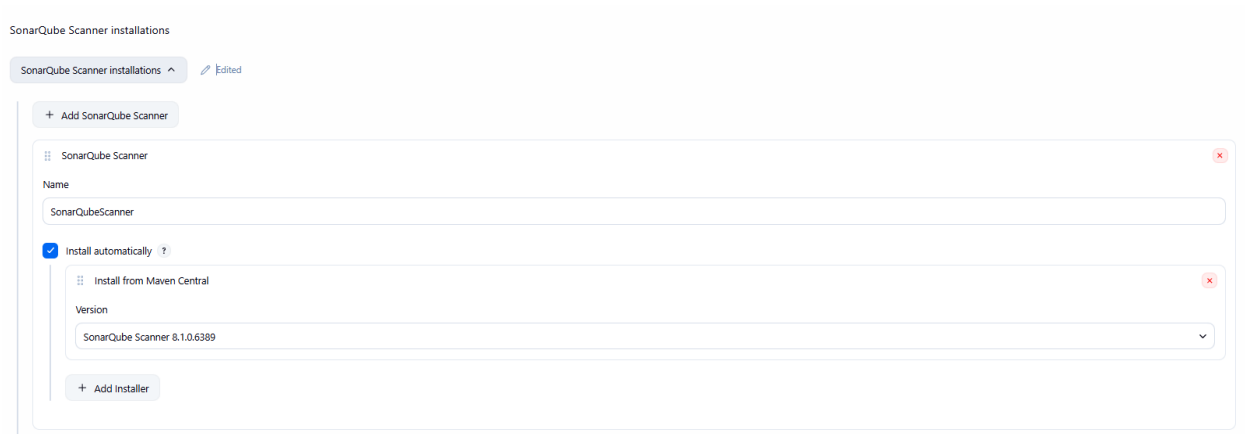


The image shows the "SonarQube servers" configuration page. It has a checkbox for "Environment variables" which is unchecked. Below that, the "SonarQube installations" section is active, showing a list of installations. One installation is visible:

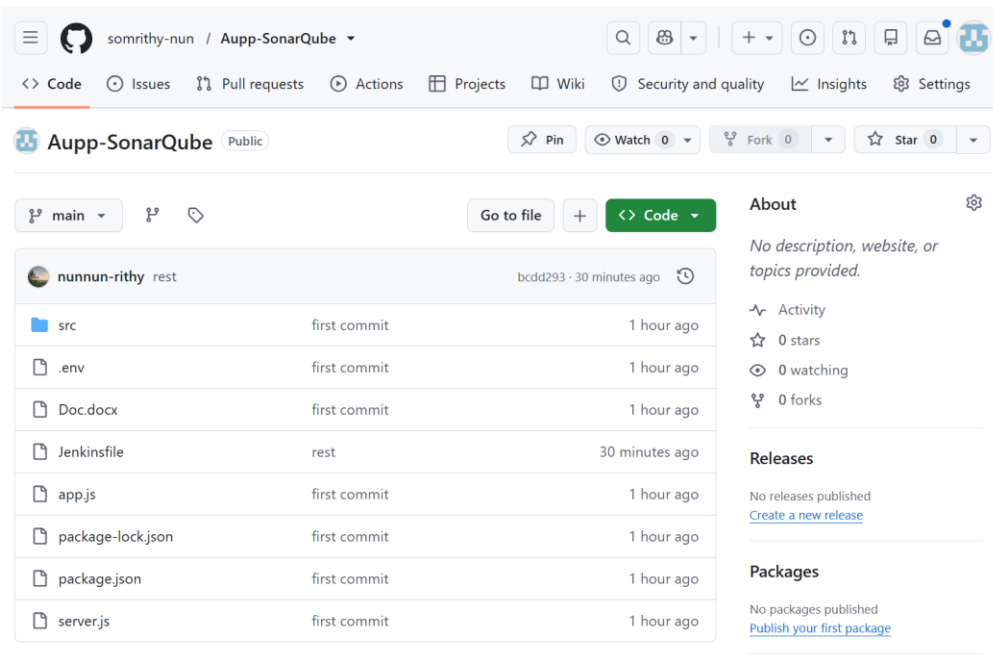
Name
SonarQube_Server_Dev

Below the table, there are fields for "Server URL" (with a default of http://localhost:9000) and "Server authentication token" (with a note that it's mandatory when anonymous access is disabled). There's a dropdown menu for "Sonar-token" and an "Add" button. At the bottom, there's an "Advanced" dropdown menu.

Task 11: Configure SonarQube Scanner Inside Jenkins – Screenshot



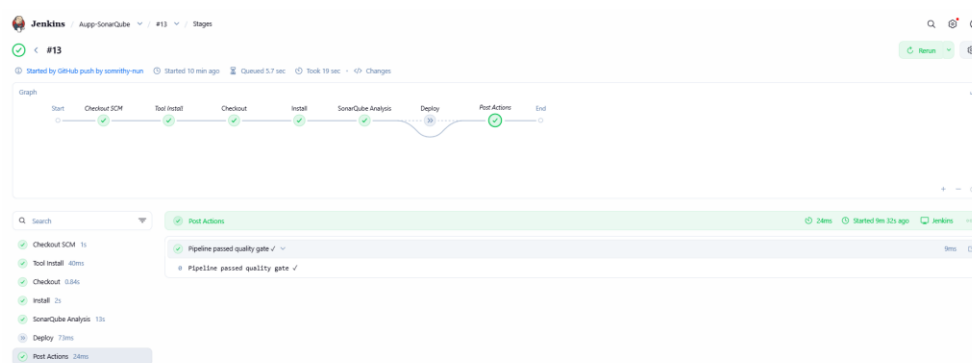
Task 12: PUSH/UPLOAD your Project Code in the GitHub



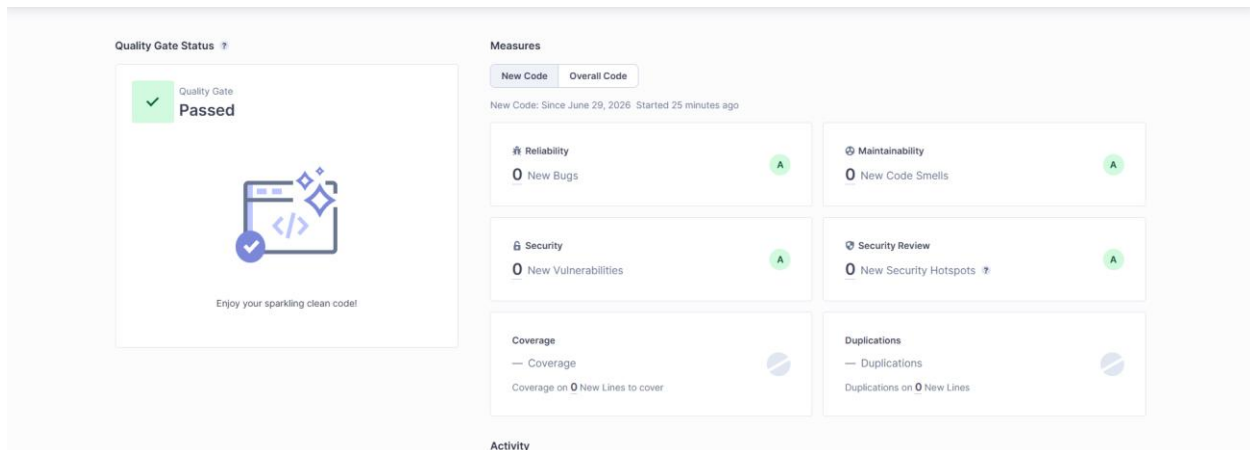
Task 13-14: Create Declarative Pipeline.

```
1 pipeline {
2   agent any
3
4   environment {
5     SONAR_HOST_URL = 'https://sonar.gravastar.store'
6     SONAR_PROJECT = 'express-app'
7   }
8
9   tools {
10    nodejs 'node-22.11.0' // must match the name in Jenkins > Global Tool Config
11  }
12
13  stages {
14    stage('Checkout') {
15      steps {
16        checkout scm
17      }
18    }
19
20    stage('Install') {
21      steps {
22        sh 'npm ci'
23      }
24    }
25
26    stage('SonarQube Analysis') {
27      steps {
28        withSonarQubeEnv('SonarQube_Server_Dev') { // must match the name in Jenkins > Configure System
29          sh """
30            npx sonar-scanner \
31              -Dsonar.projectKey=${SONAR_PROJECT} \
32              -Dsonar.sources=src \
33              -Dsonar.exclusions=node_modules/**,coverage/** \
34              -Dsonar.javascript.lcov.reportPaths=coverage/lcov.info \
35              -Dsonar.host.url=${SONAR_HOST_URL} \
36              -Dsonar.login=${SONAR_TOKEN}
37          """
38        }
39      }
40    }
41
42    stage('Deploy') {
43      when {
44        branch 'main'
45      }
46      steps {
47        sh 'echo "Add your deploy script here"'
48      }
49    }
50  }
51 }
```

Task 15: Manually build and check if it's working or not.



Task 16: Show SonarQube Report using the generated link



Task 18: Manually build and check if it's working or not. -

```
INFO: Analysis report compressed in 20ms, zip size=33.1 kB
INFO: Analysis report uploaded in 54ms
INFO: ANALYSIS SUCCESSFUL, you can find the results at: https://sonar.gravastar.store/dashboard?id=express-app
INFO: Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
INFO: More about the report processing at https://sonar.gravastar.store/api/ce/task?id=AZ8SwSXX5Ctr9ZHc-qni
INFO: Analysis total time: 10.792 s
INFO: -----
INFO: EXECUTION SUCCESS
INFO: -----
INFO: Total time: 12.440s
INFO: Final Memory: 23M/88M
INFO: -----
[Pipeline] }
[Pipeline] // withSonarQubeEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
Stage "Deploy" skipped due to when conditional
[Pipeline] getContext
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline passed quality gate ✓
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```